

**Hackathon Project Phases Template** that ensures students can complete it efficiently while covering all six phases. The template is structured to capture essential information without being time-consuming.

---

# Hackathon Project Phases Template

## Project Title:

AI's ethical simulator

## Team Name:

Mistral minds

## Team Members:

- Likhita battula
  - Susmitha Alla
  - Kalyani Chinta
- 

## Phase-1: Brainstorming & Ideation

### Objective:

The problem statement is Ethical decision-making is complex and biased, with no real-time analysis. Traditional methods lack adaptability and structured evaluation. An AI-powered tool can generate scenarios and assess choices using ethical frameworks. This helps policy-makers and businesses make informed, unbiased decisions efficiently.

- Purpose of the project is:
- The AI Ethics Simulator helps policy-makers and businesses make informed, unbiased decisions by generating ethical scenarios and analyzing consequences using AI.
- 
- The impact of project is:
- 
- It promotes fairness, transparency, and accountability, reducing biases and improving ethical decision-making across industries.

## Key Points:

### 1. Problem Statement:

Ethical decision-making is complex and biased, lacking real-time analysis. An AI-powered tool is needed to generate scenarios and provide unbiased insights.

### 2. Proposed Solution:

The AI Ethics Simulator is an AI-powered tool that generates ethical dilemmas, analyzes decision paths, and evaluates choices using ethical frameworks. It helps policy-makers and businesses make unbiased, informed decisions by providing real-time ethical insights.

### 3. Target Users:

Target Users:

Policy-makers & Businesses

Healthcare & Legal Professionals

Educators & Researchers

Beneficiaries:

Organizations & Professionals

Society as a Whole

### 4. Expected Outcome:

The AI Ethics Simulator will improve decision-making by providing real-time, unbiased ethical insights, reducing biases, and promoting fairness.

---

# Phase-2: Requirement Analysis

## Objective:

- Define technical and functional requirements.
- Technical Requirements:
- Uses Mistral AI, React.js, FastAPI, and PostgreSQL/MongoDB, hosted on AWS/GCP with reinforcement learning.
- Functional Requirements:
- Generates ethical scenarios, analyzes decisions, and provides real-time insights using ethical frameworks.

## Key Points:

### 1. Technical Requirements:

Languages: Python, JavaScript

Frontend: React.js, HTML, CSS

Backend: FastAPI, Flask

Database: PostgreSQL, MongoDB

AI & ML: Mistral AI, NLP, Reinforcement Learning

Cloud & Deployment: AWS, GCP, Docker, Kubernetes

### 2. Functional Requirements:

Functional Requirements:

Scenario Generation: AI creates ethical dilemmas based on user inputs.

Decision Analysis: Evaluates multiple decision paths using ethical frameworks.

User Interaction: Allows users to explore choices and

### 3. Constraints & Challenges:

Constraints:

AI accuracy, resources, and data privacy.

Challenges:

Ethical complexity, AI bias, and scalability.

---

## Phase-3: Project Design

### Objective

### Architecture:

**Frontend (React.js), Backend (FastAPI), AI (Mistral AI), Database (PostgreSQL/MongoDB), Cloud (AWS/GCP).**

### User Flow:

**User inputs → AI generates scenarios → Analyzes decisions → User explores → Final decision.**

### Key Points:

1. **System Architecture Diagram:**
2. Here's the simple flowchart in text format:
- 3.
4. User Input

↓

AI Generates Scenarios

↓

AI Analyzes Decisions

↓

User Explores Choices

↓

Final Decision

This represents the flow of how the AI Ethics Simulator processes decisions step by step. Let me know if you need modifications!

5. **User Flow:**
  1. Login/Register → User signs in.
  2. Enter Scenario → Inputs an ethical dilemma.

3. AI Analysis → Generates choices and insights.
  4. User Exploration → Compares and refines options.
  5. Final Decision → Selects the best ethical choice.
  6. **UI/UX Considerations:**
    - Simple UI → Easy navigation.
    - Visual Insights → Graphs & charts.
    - Real-Time Feedback → Instant analysis.
    - Accessibility → Inclusive design.
    - Dark/Light Mode → Custom themes.
- 

## Phase-4: Project Planning (Agile Methodologies)

### Objective:

- Sprint 1: Define scope & research.
- Sprint 2: Design UI/UX.
- Sprint 3: Develop frontend & backend.
- Sprint 4: Integrate AI.
- Sprint 5: Test & deploy.

### Key Points:

#### 1. Sprint Planning:

Sprint 1: Research & define requirements.

Sprint 2: Design UI/UX wireframes.

Sprint 3: Develop frontend & backend.

Sprint 4: Integrate AI & database.

Sprint 5: Testing, debugging, and deployment.

#### 2. Task Allocation:

Manager: Oversees progress.

UI/UX Designer: Creates UI/UX.

Frontend Dev: Builds UI.

Backend Dev: Develops API.

AI Engineer: Integrates AI.

QA Tester: Tests & debugs.

DevOps: Manages deployment.

3. **Timeline & Milestones:**

4. Week 1-2: Research & planning.
  5. Week 3-4: UI/UX design
  6. Week 5-7: Develop frontend & backend.
  7. Week 8-9: AI integration.
  8. Week 10-12: Testing & deployment.
-

# Phase-5: Project Development

## Objective:

Project structure

- ethical-decision-sim/
  - | — backend/ # FastAPI Backend
    - | | — main.py # API logic
    - | | — database.py # PostgreSQL setup
    - | | — ai\_engine.py # Mistral AI Integration
  - | — frontend/ # React Frontend
    - | | — src/
      - | | | — components/
        - | | | | — Home.js
        - | | | | — DecisionForm.js
      - | | | — App.js
  - | — docker-compose.yml # Deployment Config
  - | — README.md # Documentation
- Install dependencies
- `pip install fastapi uvicorn pydantic psycpg2 sqlalchemy requests`
- Backend code
- `from fastapi import FastAPI`
- `from pydantic import BaseModel`
- `from database import save_decision`
- `from ai_engine import analyze_ethics`
- `import random`
- 
- `app = FastAPI()`

- 
- class DecisionRequest(BaseModel):
- question: str
- 
- @app.post("/generate\_scenario/")
- def generate\_scenario(request: DecisionRequest):
- scenarios = [
- f"Scenario 1: {request.question} - Option A leads to outcome X, Option B to Y.",
- f"Scenario 2: {request.question} - Ethical principle Z should be considered."
- ]
- analysis = analyze\_ethics(request.question)
- save\_decision(request.question, analysis)
- return {"scenario": random.choice(scenarios), "analysis": analysis}
- Database
- from sqlalchemy import create\_engine, Column, String, Integer
- from sqlalchemy.ext.declarative import declarative\_base
- from sqlalchemy.orm import sessionmaker
- 
- DATABASE\_URL = "postgresql://user:password@localhost/ethicaldb"
- engine = create\_engine(DATABASE\_URL)
- SessionLocal = sessionmaker(bind=engine)
- Base = declarative\_base()
- 
- class Decision(Base):
- \_\_tablename\_\_ = "decisions"
- id = Column(Integer, primary\_key=True, index=True)



- `question = Column(String)`
- `scenario = Column(String)`
- 
- `def save_decision(question, scenario):`
- `session = SessionLocal()`
- `decision = Decision(question=question, scenario=scenario)`
- `session.add(decision)`
- `session.commit()`
- `session.close()`
- AI integration
- `import requests`
- 
- `MISTRAL_API_URL = "https://mistral-ai-api.com/analyze"`
- 
- `def analyze_ethics(question):`
- `response = requests.post(MISTRAL_API_URL, json={"question": question})`
- `return response.json().get("analysis", "No analysis available.")`
- Frontend
- `npx create-react-app frontend`
- `cd frontend`
- `npm install axios react-router-dom`
- Frontend code
- `import React from "react";`
- `import { BrowserRouter as Router, Route, Routes } from "react-router-dom";`
- `import Home from "../components/Home";`
- `import DecisionForm from "../components/DecisionForm";`

- 
- function App() {
- return (
- <Router>
- <Routes>
- <Route path="/" element={<Home />} />
- <Route path="/decision" element={<DecisionForm />} />
- </Routes>
- </Router>
- );
- }
- 
- export default App;
- 
- Homepage
- import React from "react";
- import { Link } from "react-router-dom";
- 
- function Home() {
- return (
- <div>
- <h1>Ethical Decision-Making Simulator</h1>
- <Link to="/decision">Start Decision Process</Link>
- </div>
- );
- }
-

- export default Home;
- Decision form
- import React, { useState } from "react";
- import axios from "axios";
- 
- function DecisionForm() {
- const [question, setQuestion] = useState("");
- const [result, setResult] = useState(null);
- 
- const handleSubmit = async (e) => {
- e.preventDefault();
- const response = await axios.post("http://localhost:8000/generate\_scenario/", {
- question });
- setResult(response.data);
- };
- 
- return (
- <div>
- <h2>Enter an Ethical Dilemma</h2>
- <form onSubmit={handleSubmit}>
- <input type="text" value={question} onChange={(e) => setQuestion(e.target.value)} required />
- <button type="submit">Generate Scenario</button>
- </form>
- {result && <div><h3>Scenario:</h3> <p>{result.scenario}</p></div>}
- </div>
- );

- }
- 
- export default DecisionForm;
- Run postgres SQL and create database
- sudo -u postgres psql
- CREATE DATABASE ethicaldb;
- CREATE USER user WITH ENCRYPTED PASSWORD 'password';
- GRANT ALL PRIVILEGES ON DATABASE ethicaldb TO user;
- Run database migrations
- python -c "from database import Base, engine; Base.metadata.create\_all(engine)"
- Deployment with docker
- version: "3.8"
- 
- services:
- backend:
- build: ./backend
- ports:
- - "8000:8000"
- depends\_on:
- - db
- 
- frontend:
- build: ./frontend
- ports:
- - "3000:3000"
- depends\_on:

- - backend
- 
- db:
- image: postgres
- environment:
- POSTGRES\_USER: user
- POSTGRES\_PASSWORD: password
- POSTGRES\_DB: ethicaldb
- ports:
- - "5432:5432"
- Run docker
- docker-compose up --build
- Hosting & Deployment
- 
- Frontend: Deploy on Vercel or Netlify
- 
- Backend: Deploy on AWS EC2 or GCP Cloud Run
- 
- Database: Use AWS RDS or Google Cloud SQL

## Key Points:

### 1. Technology Stack Used:

Tech Stack

Frontend: React.js, Axios, React Router

Backend: FastAPI, SQLAlchemy, Requests

Database: PostgreSQL

AI: Mistral AI

Deployment: Docker, AWS/GCP, Vercel/Netlify

2. **Development Process:**

Plan → Build UI → Develop API → Set Up DB → Test & Deploy

3. **Challenges & Fixes:**

4. AI tuning, scaling, DB optimization, security, UI improvements.
- 

## Phase-6: Functional & Performance Testing

### Objective:

- Test components, verify integration, gather feedback, optimize, secure.

### Key Points:

1. **Test Cases Executed:** (List the scenarios tested)
  2. **Bug Fixes & Improvements:** (Mention fixes made)
  3. **Final Validation:** (Does the project meet the initial requirements?)
  4. **Deployment (if applicable):** (Hosting details or final demo link)
- 

## Final Submission

1. **Project Report Based on the templates**
  2. **Demo Video (3-5 Minutes)**
  3. **GitHub/Code Repository Link**
  4. **Presentation**
-